CORE COMPUTER SCIENCE

A practical study guide for interviews and fundamentals

---

01 Operating Systems

02 Computer Networking

03 Threads & Concurrency

04 DBMS & SQL

05 Python Basics

Minimal black-and-white edition • Concepts, comparisons, examples, and practice questions

How to use this guide: read the concept, explain it aloud in your own words, then answer the review questions without looking. Treat examples as starting points, not as a substitute for coding or experiments.

How to use this guide

This guide is designed as a compact but broad foundation. Each section moves from the mental model to the mechanism, common trade-offs, and interview-style questions.

Suggested study loop

- First pass: read definitions and comparison tables.
- Second pass: draw the flow or lifecycle from memory.
- Third pass: solve the practice questions and explain why alternatives are wrong.
- For coding topics, run small examples and inspect edge cases.

Contents

Section	Topics
1. Operating Systems	Processes, threads, scheduling, memory, virtual memory, files, I/O, system calls, deadlocks
2. Networking	Layers, Ethernet/IP, TCP/UDP, DNS, HTTP, TLS, routing, sockets, proxies, troubleshooting
3. Threads & Concurrency	Race conditions, locks, atomics, deadlocks, condition variables, pools, async, GIL
4. DBMS & SQL	Relational model, keys, normalization, indexes, transactions, ACID, isolation, joins, query plans
5. Python	Syntax, data types, control flow, functions, collections, OOP, exceptions, modules, iterators, testing
6. Revision	Quick comparisons, common interview prompts, practice checklist

1. Operating Systems

An operating system manages hardware resources and provides abstractions—processes, virtual memory, files, and sockets—so programs can run safely and efficiently.

1.1 Kernel, user mode, and system calls

- **Kernel:** privileged core that manages CPU scheduling, memory, devices, and protection.
- **User mode:** applications run with restricted privileges. A faulting or malicious app should not directly control hardware.
- **System call:** controlled entry into the kernel, for example opening a file, creating a process, or sending data on a socket.
- A library call is not automatically a system call. A library may perform work in user space and invoke a system call only when needed.

1.2 Process vs thread

Concept	Process	Thread
Meaning	Running program with its own virtual address space and resources	Execution path within a process
Memory	Usually isolated from other processes	Threads in one process share heap and globals
Communication	IPC: pipes, sockets, shared memory, queues	Shared variables; synchronization required
Cost	Creation/context switch often more expensive	Often cheaper, but not free
Failure	Usually isolated from other processes	A bad thread can corrupt or terminate its process

Context switch: the OS saves the current execution state and restores another. Switching may involve register state, scheduling decisions, cache effects, and address-space changes.

1.3 Process lifecycle

A common model is **new** → **ready** → **running** → **waiting/blocked** → **ready** → **terminated**. A running process may be preempted and returned to the ready queue. Waiting means it cannot proceed until an event, such as I/O completion, occurs.

1.4 CPU scheduling

Algorithm	Idea	Strength / weakness
FCFS	Run in arrival order	Simple; convoy effect and poor response time possible
SJF / SRTF	Shortest job first / shortest remaining time	Good average waiting time if durations are known; long jobs may starve
Round Robin	Each runnable task gets a time slice	Responsive for interactive workloads; tiny slices cause overhead
Priority	Higher-priority work first	Low-priority starvation; aging can help
Multilevel feedback queue	Move tasks between queues based on behavior	Flexible, but tuning is complex

- Throughput = completed work per unit time. Turnaround time = completion – arrival. Waiting time = time spent ready. Response time = first run – arrival.
- Preemptive scheduling can interrupt a running task; non-preemptive scheduling waits until it yields or blocks.

1.5 Concurrency and deadlocks

- **Race condition:** output depends on timing/interleaving of concurrent operations.
- **Critical section:** code accessing shared state that must obey a synchronization rule.

- **Deadlock:** tasks wait forever for resources held by one another.

The four classic deadlock conditions are **mutual exclusion**, **hold and wait**, **no preemption**, and **circular wait**. Prevent deadlock by breaking at least one condition; avoid it with safe-state reasoning; detect and recover; or accept the risk where appropriate.

1.6 Memory and virtual memory

- **Virtual address space:** the address view a process uses. The MMU and page tables map virtual pages to physical frames.
- **Page:** fixed-size virtual-memory unit. **Page fault:** translation or access cannot be completed immediately; it may be a normal demand-page event or an invalid access.
- **TLB:** cache of recent address translations. A TLB miss may require a page-table walk; it is not necessarily a page fault.
- **Demand paging:** load pages when accessed. If physical memory is pressured, pages may be evicted; excessive paging can cause thrashing.
- **Page replacement:** policies include FIFO, LRU approximations, and Clock. The best policy depends on workload and system design.
- **Stack:** commonly holds call frames and local execution state. **Heap:** commonly holds dynamically allocated objects. Exact placement is language/runtime dependent.

1.7 Memory allocation and protection

- Contiguous allocation can suffer external fragmentation; paging avoids external fragmentation in physical placement but can waste space inside the final page.
- Segmentation divides memory by logical regions; paging divides it into fixed-size blocks. Systems can combine both ideas.
- Memory protection uses permissions such as read/write/execute. Address-space isolation limits accidental or malicious access across processes.

1.8 Filesystems and I/O

- A file system maps names and directories to metadata and data blocks. Metadata can include size, ownership, permissions, and timestamps.
- A file descriptor is a process-local handle to an open file, socket, pipe, or similar resource. It is not the file itself.
- Buffered I/O may reduce system calls; direct or asynchronous I/O can be useful for specific performance requirements.
- Interrupts notify the CPU of events; polling repeatedly checks for them. DMA allows devices to transfer data to/from memory with less CPU involvement.

1.9 IPC and virtualization

- Pipes/FIFOs stream bytes; message queues preserve message boundaries; shared memory is fast but needs synchronization; sockets work locally or across machines.
- A virtual machine virtualizes hardware and runs a guest OS. A container generally isolates processes while sharing the host kernel.

OS interview questions

- What happens from the moment a program is launched until it executes?
- Why can threads be faster than processes, and what risks do they introduce?
- Explain a page fault, TLB miss, and context switch without conflating them.
- How would you diagnose high CPU, high memory, and high I/O separately?
- Give a real example of deadlock and one way to prevent it.

2. Computer Networking

Networking moves data between programs and machines. A useful mental model is layered protocols: each layer offers a service to the layer above and relies on the layer below.

2.1 Layers and encapsulation

TCP/IP layer	Examples	Responsibility
Application	HTTP(S), DNS, SMTP, SSH	Application-level meaning and message formats
Transport	TCP, UDP	Process-to-process delivery, ports, reliability or datagrams
Internet	IP, ICMP	Addressing and routing packets between networks
Link	Ethernet, Wi-Fi, ARP/neighbor discovery	Delivery over a local link

Encapsulation adds protocol headers as data moves down the stack; receiving layers remove and interpret them. A frame carries a packet, which carries a transport segment/datagram, which carries application data.

2.2 IP addresses, ports, and routing

- An IP address identifies a network interface/address in an IP network; IPv4 uses 32 bits and IPv6 uses 128 bits.
- A subnet prefix divides network and host portions. A router forwards packets between networks using routing tables.
- A port identifies a transport endpoint on a host. A connection is commonly distinguished by protocol plus source/destination IP addresses and ports.
- NAT translates address/port information between address realms; it is common in IPv4 networks but is not the same as routing.
- DNS maps names to records such as A/AAAA addresses, MX mail servers, and CNAME aliases. Results may be cached according to TTL.

2.3 TCP vs UDP

Property	TCP	UDP
Connection	Connection-oriented byte stream	Connectionless datagrams
Reliability	Sequence numbers, acknowledgements, retransmission	No built-in delivery guarantee
Ordering	Ordered byte stream	Datagrams may arrive out of order
Flow/congestion control	Built in	Application must handle if needed
Use cases	Web, SSH, databases, file transfer	DNS, voice/video, games, QUIC transport substrate

TCP provides a reliable ordered byte stream, not application message boundaries. Applications must frame messages—for example with lengths, delimiters, or a structured protocol.

2.4 TCP connection lifecycle

- A typical TCP handshake is SYN → SYN-ACK → ACK. It establishes sequence-number state and confirms reachability in both directions.
- TCP uses sequence numbers and acknowledgements to track data, retransmits lost data, and uses a congestion window to adapt to network capacity.
- Closing may use FIN/ACK exchanges; RST aborts a connection. TIME_WAIT helps handle delayed duplicate segments and orderly connection teardown.

2.5 HTTP, HTTPS, and TLS

- HTTP is an application protocol. A request includes a method, target, headers, and optionally a body; a response includes status, headers, and optionally a body.
- Common methods: GET retrieves, POST submits/processes, PUT replaces, PATCH partially updates, DELETE removes. Idempotency means repeating an operation has the same intended effect as doing it once.
- Status families: 2xx success, 3xx redirection, 4xx client-side issue, 5xx server-side issue. Common codes: 200, 201, 204, 301/302, 304, 400, 401, 403, 404, 409, 429, 500, 502, 503, 504.
- HTTPS is HTTP protected by TLS. TLS authenticates the server via certificates and negotiates encryption/integrity; certificate validation and hostname checks matter.
- HTTP/1.1 commonly uses persistent connections; HTTP/2 multiplexes streams over a connection; HTTP/3 uses QUIC over UDP. Protocol choice does not remove application-level latency or bottlenecks.

2.6 Latency, throughput, and reliability

- Latency is elapsed time for an operation; throughput is completed work per unit time; bandwidth is link capacity. A high-bandwidth link can still have high latency.
- A rough request latency budget includes DNS lookup, connection setup, TLS negotiation, server processing, queueing, and data transfer. Caching and connection reuse can reduce repeated setup cost.
- Packet loss can trigger retransmissions or reduce throughput. Congestion occurs when offered load exceeds available capacity; queues may increase latency before packets are dropped.

2.7 Network components

Component	Purpose
Switch	Forwards frames within a LAN, typically using MAC address learning
Router	Forwards IP packets between networks
Load balancer	Distributes traffic across backends; may operate at L4 or L7
Forward proxy	Acts on behalf of clients
Reverse proxy	Sits in front of servers; may terminate TLS, route, cache, or enforce policy
Firewall	Allows or blocks traffic according to rules and connection state
CDN	Caches/serves content closer to users and can absorb traffic spikes

2.8 Sockets and troubleshooting

- A socket is an OS abstraction for network communication. Servers typically bind, listen, accept, then read/write; clients resolve, connect, and exchange data.
- Use a layered debugging approach: DNS resolution → route/reachability → TCP connection → TLS/certificate → HTTP status → application logs/dependencies.
- Useful distinctions: connection refused often means no listener or an active reject; timeout means no timely response; DNS failure happens before connecting; 502/503/504 usually involve gateway or upstream behavior.

Networking interview questions

- Walk through what happens after entering a URL in a browser.
- Why does TCP need sequence numbers, acknowledgements, and congestion control?
- Explain DNS caching, TTL, and what can happen after an IP address changes.
- Compare a reverse proxy, load balancer, and CDN.
- How would you distinguish a DNS issue from a TLS issue or a server timeout?

3. Threads & Concurrency

Concurrency is about overlapping progress; parallelism is about simultaneous execution. Correct concurrent code must control shared state and define what other threads are allowed to observe.

3.1 Core terms

- **Concurrency:** multiple tasks make progress during overlapping periods, potentially by interleaving.
- **Parallelism:** tasks execute at the same instant on multiple execution units.
- **Race condition:** result depends on uncontrolled timing. A data race is a specific unsynchronized conflicting access, in languages/memory models that define the term.
- **Atomicity:** an operation appears indivisible. **Visibility:** one thread's write becomes observable by another. **Ordering:** operations are observed in a permitted order.

3.2 Locks and synchronization

Primitive	Use	Caution
Mutex/lock	Protect a critical section	Keep scope small; consistent lock ordering
Semaphore	Control access to N resources or signal permits	Permit accounting errors can cause bugs
Condition variable	Wait until a predicate becomes true	Wait in a loop and re-check the predicate
Read-write lock	Many readers or exclusive writer	Writer starvation and overhead possible
Atomic operation	Small lock-free state transitions	Memory ordering and algorithm correctness are subtle
Barrier	Wait until a group reaches a phase	One missing participant can block everyone

3.3 Why incrementing is not necessarily atomic

```
counter = counter + 1
```

Conceptually this can be read → add → write. If two threads both read 10 before either writes, both may write 11 and one increment is lost. Use a lock or an appropriate atomic primitive; do not assume a single source-code line is indivisible.

3.4 Deadlock, livelock, and starvation

- **Deadlock:** tasks wait indefinitely for one another. A consistent global lock order is a common prevention technique.
- **Livelock:** tasks keep reacting to each other but make no useful progress.
- **Starvation:** a task is repeatedly denied resources or scheduling time.
- Avoid calling unknown callbacks while holding locks. Avoid acquiring locks in inconsistent order. Use timeouts where meaningful, but do not treat timeouts as proof of correctness.

3.5 Producer-consumer pattern

A producer creates work and places it in a bounded queue; consumers remove and process it. The queue provides coordination and backpressure. If full, producers may block or reject; if empty, consumers may wait.

```
while queue_is_empty():
    condition.wait()
item = queue.pop()
```

The predicate must be checked in a loop because wake-ups can be spurious and another thread may change the condition before the awakened thread reacquires the lock.

3.6 Thread pools and task design

- A thread pool reuses worker threads and bounds concurrency. It reduces repeated creation overhead and helps cap resource usage.

- Too few workers limit throughput; too many can cause context-switching, memory use, lock contention, or downstream overload.
- Separate CPU-bound and I/O-bound work when their needs differ. Use bounded queues to prevent unbounded memory growth.

3.7 Async I/O vs threads vs processes

Approach	Good fit	Trade-off
Threads	Blocking I/O, shared-memory tasks, moderate concurrency	Synchronization and shared-state complexity
Processes	CPU-bound parallel work or stronger isolation	IPC and memory overhead
Async/event loop	Many I/O-bound tasks with async-compatible libraries	Blocking calls stall the loop; requires async-aware design

3.8 Python and the GIL

- In standard CPython builds, the Global Interpreter Lock (GIL) historically permits only one thread at a time to execute Python bytecode in a given interpreter. This often limits CPU-bound pure-Python thread parallelism.
- Threads can still help I/O-bound workloads because they can wait on I/O while other work proceeds. Multiprocessing can use multiple processes for CPU parallelism, with serialization/IPC costs.
- Python has evolving free-threaded builds and implementation details vary by version/build. Avoid treating the GIL as a universal property of every Python implementation.

Concurrency interview questions

- What is the difference between concurrency and parallelism?
- Show how a race condition can happen and how you would fix it.
- How can deadlock occur with two locks? Give a prevention strategy.
- When would you choose asyncio, threads, or multiprocessing in Python?
- Why should a condition variable be used with a predicate loop?

4. DBMS & SQL

A database management system stores and retrieves data while providing constraints, concurrency control, crash recovery, and query optimization. Good schema design and indexes should reflect actual access patterns.

4.1 Relational fundamentals

- A relation is represented as a table of rows and columns. A schema defines structure and constraints; a query expresses what data is wanted, not necessarily how to retrieve it.
- **Primary key:** uniquely identifies a row and cannot be NULL. **Candidate key:** a minimal unique key. **Foreign key:** enforces a relationship to referenced data.
- **Unique constraint:** prevents duplicate key values, with NULL behavior varying by database. **NOT NULL**, **CHECK**, and foreign-key constraints enforce data quality.

4.2 Normalization

Form	Practical meaning
1NF	Values are atomic under the model; avoid repeating groups in a single field
2NF	1NF plus no partial dependency of a non-key attribute on part of a composite candidate key
3NF	No inappropriate transitive dependency of non-key attributes on keys
BCNF	Every non-trivial functional dependency has a superkey as its determinant

Normalization reduces duplication and update anomalies. Denormalization may improve selected read paths, but adds consistency and write complexity.

4.3 Indexes

- An index is an auxiliary structure that helps locate rows without scanning the whole table. B-tree indexes are common for equality, range, and ordered access.
- A hash index can be useful for equality lookups where supported, but typically does not support range ordering like a B-tree.
- Indexes consume disk and memory and add write/update/delete cost. Index the columns used by selective filters, joins, and ordering when the workload benefits.
- A composite index on (A, B) commonly helps predicates starting with A; it may not efficiently support a predicate on B alone. Exact behavior depends on engine and index type.
- A covering index contains all data needed by a query, potentially avoiding table lookups. Too many or wide indexes can harm writes and cache efficiency.

4.4 Transactions and ACID

Property	Meaning
Atomicity	A transaction commits as a unit or its changes are rolled back
Consistency	A transaction preserves declared constraints and application invariants
Isolation	Concurrent transactions are controlled to meet the chosen isolation level
Durability	Committed changes survive failures within the system's guarantees

A transaction groups logically related operations. Commit makes changes durable according to the database's configuration and guarantees; rollback abandons uncommitted work.

4.5 Isolation anomalies

Anomaly	Meaning
Dirty read	Read data written by a transaction that has not committed
Non-repeatable read	Re-read a row and see a different committed value
Phantom read	Repeat a predicate query and see a changed set of matching rows
Lost update	Concurrent writes overwrite one another's changes
Write skew	Transactions read overlapping state and update different rows, violating a shared invariant

- Isolation labels and exact behavior differ by database. Snapshot isolation can prevent some anomalies but may permit write skew; serializable isolation aims to produce an outcome equivalent to some serial execution.

4.6 Locks, MVCC, and recovery

- Lock-based control may use shared/read and exclusive/write locks. Lock granularity can range from rows to pages or tables.
- Two-phase locking grows a lock set before releasing locks; strict variants hold write locks until commit/abort. Locking can block and deadlock.
- MVCC (multi-version concurrency control) keeps versions so readers can often access a consistent snapshot without blocking writers. Cleanup/vacuuming of old versions is important in systems that use it.
- Write-ahead logging records changes before the corresponding data pages are written, enabling recovery after a crash. Checkpoints limit recovery work.

4.7 SQL essentials

```
SELECT department, COUNT(*) AS headcount
FROM employees
WHERE active = TRUE
GROUP BY department
HAVING COUNT(*) >= 5
ORDER BY headcount DESC;
```

Logical query processing is commonly taught as FROM/JOIN → WHERE → GROUP BY → HAVING → SELECT → DISTINCT → ORDER BY → LIMIT. The optimizer may execute a different physical plan.

Operation	Use
INNER JOIN	Only matching row pairs
LEFT JOIN	All left rows plus matches; unmatched right columns become NULL
RIGHT JOIN	All right rows plus matches
FULL OUTER JOIN	All rows from both sides, matched where possible
CROSS JOIN	Cartesian product
UNION	Combine compatible result sets and remove duplicates
UNION ALL	Combine result sets without duplicate elimination

4.8 NULL, aggregation, and subqueries

- NULL means unknown/missing, not zero or an empty string. Use IS NULL / IS NOT NULL; comparisons like = NULL do not evaluate to true.
- COUNT(*) counts rows; COUNT(column) counts non-NULL values. SUM/AVG typically ignore NULL inputs. GROUP BY creates groups for aggregation.
- A WHERE filter acts before grouping; HAVING filters groups after aggregation.
- EXISTS checks whether a subquery returns rows. A correlated subquery refers to the outer row; the optimizer may transform it into a join or other plan.

4.9 Query plans and performance

- Use EXPLAIN (and, where safe, execution-analysis tools) to inspect scans, index use, join methods, estimated rows, actual rows, and costly sorts.
- A sequential/table scan is not inherently bad: it can be optimal for small tables or queries returning a large fraction of rows.
- Performance diagnosis: measure first; inspect slow queries; check cardinality estimates and indexes; reduce unnecessary data transfer; consider query shape, locking, and connection-pool saturation.

4.10 SQL examples

```
-- Find customers with no orders
SELECT c.id, c.name
FROM customers c
LEFT JOIN orders o ON o.customer_id = c.id
WHERE o.id IS NULL;
```

```
-- Latest order per customer (window function)
SELECT *
FROM (
  SELECT o.*,
         ROW_NUMBER() OVER (
           PARTITION BY customer_id
           ORDER BY created_at DESC
         ) AS rn
  FROM orders o
) ranked
WHERE rn = 1;
```

```
-- Running total
SELECT account_id, posted_at, amount,
       SUM(amount) OVER (
         PARTITION BY account_id
         ORDER BY posted_at
       ) AS running_total
FROM transactions;
```

DBMS interview questions

- When does a composite index help, and why does column order matter?
- Explain ACID with a concrete money-transfer example.
- Compare read committed, repeatable read, and serializable at a high level.
- What is the difference between WHERE and HAVING?
- How would you investigate a slow query without blindly adding indexes?

5. Python Basics

Python emphasizes readable syntax and dynamic typing. Learn the object model, collection behavior, function semantics, exceptions, and iteration—not just syntax.

5.1 Names, objects, and types

- A variable name refers to an object; assignment binds a name rather than copying an object automatically.
- Python is dynamically typed: types belong to objects, and names can be rebound to objects of different types.
- Mutable examples: list, dict, set. Immutable examples: int, float, bool, str, tuple (though a tuple can contain mutable objects).

```
x = [1, 2]
y = x          # both names refer to the same list
y.append(3)
# x is now [1, 2, 3]
z = x.copy()   # shallow copy
```

5.2 Core built-in collections

Type	Properties	Typical use
list	Ordered, mutable, allows duplicates	Sequences you update
tuple	Ordered, immutable container	Fixed records, hashable if all elements are hashable
dict	Key-value mapping, insertion-ordered	Fast average-case key lookup
set	Unique hashable elements	Membership and set algebra
str	Immutable Unicode text sequence	Text processing

- Membership: `x in list` is generally linear; membership in a set or dictionary is average-case constant time, subject to hashing and collisions.
- Dictionary keys and set members must be hashable. Equality and hashing must obey compatible rules.

5.3 Control flow

```
for item in items:
    if item is None:
        continue
    if item == target:
        break
else:
    print('loop completed without break')
```

The loop `else` clause runs when the loop completes normally, not when terminated by `break`. Use `range` for integer sequences, `enumerate` for index-value pairs, and `zip` to iterate sequences together.

5.4 Functions and arguments

```
def greet(name, greeting='Hello'):
    return f'{greeting}, {name}'

def total(*values, tax=0.0):
    return sum(values) * (1 + tax)
```

- Arguments can be positional or keyword-based. `*args` collects extra positional arguments; `**kwargs` collects extra keyword arguments.
- Avoid mutable default arguments; defaults are evaluated once when the function is defined.

```
def add_item(item, items=None):
    if items is None:
        items = []
    items.append(item)
    return items
```

- Python passes object references by assignment. Rebinding a parameter does not rebind the caller's name; mutating a shared mutable object may be visible to the caller.

5.5 Scope and closures

- LEGB lookup: Local → Enclosing → Global → Built-in. `global` declares a global binding; `nonlocal` refers to a binding in an enclosing function scope.
- A closure can retain references to variables from an enclosing scope. Be aware of late binding when closures are created inside loops.

5.6 Comprehensions and unpacking

```
squares = [n * n for n in range(10)]
lookup = {word: len(word) for word in words}
unique = {x for x in values}
first, *middle, last = sequence
```

Comprehensions are concise transformations/filters. Prefer readability over nesting several loops or conditions into one expression.

5.7 OOP and special methods

- **Class:** defines behavior and usually shared structure. **Instance:** an object created from a class. `self` conventionally refers to the instance.
- `__init__` initializes a newly created instance; `__repr__` provides a developer-oriented representation; `__str__` provides a user-oriented string.
- Inheritance reuses/extends behavior. Composition builds objects from other objects and is often easier to change than deep inheritance.
- Use `@property` for attribute-like access to computed values; use `dataclass` for classes primarily storing data.

```
class Counter:
    def __init__(self, start=0):
        self.value = start

    def increment(self, amount=1):
        self.value += amount
        return self.value
```

5.8 Exceptions and resource handling

```
try:
    result = int(text)
except ValueError as exc:
    print(f'Invalid integer: {exc}')
else:
    print(result)
finally:
    cleanup()
```

- Catch exceptions you can handle meaningfully; avoid broad `except Exception` blocks that silently hide defects.
- Use context managers for resources. A `with` block reliably invokes cleanup behavior, including when exceptions occur.

```
with open('notes.txt', encoding='utf-8') as file:
    content = file.read()
```

5.9 Iterators, generators, and comprehensions

- An iterable can produce an iterator. An iterator returns values one at a time via `next()` and eventually raises `StopIteration`.
- A generator function uses `yield` to produce values lazily, reducing memory use when the full sequence need not be materialized.

```
def read_chunks(stream, size=4096):
    while True:
        chunk = stream.read(size)
        if not chunk:
            return
```

```
        yield chunk
```

5.10 Modules, packages, and environments

- A module is commonly a Python file; a package organizes modules. Imports execute module-level code on first import and cache the module in `sys.modules`.
- Protect script entry points with `if __name__ == '__main__':` so code runs as a script but not automatically when imported.
- Virtual environments isolate project dependencies. Pin and update dependencies deliberately; avoid committing secrets or machine-specific paths.

5.11 Typing, testing, and quality

- Type hints document expected interfaces and support static analysis; Python generally does not enforce them at runtime by default.
- Write small, deterministic functions. Test normal cases, boundary cases, empty inputs, invalid inputs, and error paths.
- Useful tools: `unittest` or pytest for tests, `pdb` for debugging, formatters/linters for consistency, and profilers for performance measurement.

5.12 Complexity reminders

Operation	Typical average complexity
List indexing	O(1)
List append	Amortized O(1)
List search / membership	O(n)
Insert/delete near list front	O(n) due to shifting
Dict/set lookup, insert, delete	Average O(1); worst case can be worse
Sorting a list	O(n log n) typical comparison-sort behavior

Python interview questions

- What is the difference between == and is?
- Why are mutable default arguments dangerous?
- What are iterables, iterators, and generators?
- Explain shallow copy versus deep copy.
- When would you use a set instead of a list?
- How do exceptions and context managers help resource safety?

6. Quick revision sheets

6.1 High-value distinctions

Pair	Remember
Process vs thread	Separate address spaces vs shared process memory
Concurrency vs parallelism	Overlapping progress vs simultaneous execution
TCP vs UDP	Reliable ordered stream vs datagrams without built-in reliability
Latency vs throughput	Time per operation vs work completed per time
Authentication vs authorization	Who are you? vs what may you do?
Primary key vs foreign key	Identifies a row vs references another key
WHERE vs HAVING	Filter rows before grouping vs filter groups
DELETE vs TRUNCATE vs DROP	Remove selected rows vs remove table rows broadly vs remove the object; details vary by DB
Thread vs async task	OS/runtime thread vs cooperatively scheduled task
Shallow vs deep copy	Copies outer container vs recursively copies nested objects
List vs tuple	Mutable sequence vs immutable sequence container

6.2 Practical debugging checklist

- Reproduce the issue and define the expected versus actual behavior.
- Collect evidence: logs, metrics, traces, error messages, and a minimal test case.
- Separate CPU, memory, I/O, network, database, and dependency bottlenecks.
- Change one variable at a time; verify the fix under realistic load and edge cases.
- Consider failure modes: timeouts, retries, duplicate requests, partial writes, stale caches, and resource exhaustion.

6.3 Practice prompts

OS. Explain how virtual memory lets processes use the same virtual address without sharing physical memory.

OS. A server is slow and CPU usage is low. Which OS and I/O causes would you investigate?

Networking. Trace a request from DNS resolution through TLS and an HTTP response.

Networking. Explain why adding bandwidth may not solve a latency problem.

Concurrency. Design a bounded producer-consumer queue and describe its shutdown behavior.

Concurrency. Explain a deadlock caused by two locks and fix it using a lock order.

DBMS. Design tables for users and orders, including keys and constraints.

DBMS. Given a slow query with a selective WHERE clause, how would you inspect the plan and choose an index?

Python. Write a function that returns the first non-repeating character in a string.

Python. Write a generator that yields lines from a file while skipping blank lines.

6.4 Short coding exercises

- Implement a stack and queue using Python collections.
- Count word frequencies in a string, ignoring case and punctuation.
- Merge two sorted lists in O(n + m) time.
- Implement an LRU cache and explain its time complexity.

- Write SQL to find duplicate email addresses.
- Write SQL to select the second-highest distinct salary.
- Use threads or asyncio to fetch several independent resources, and compare the designs.
- Create a transaction-safe transfer between two accounts and list the required invariants.

Final study checklist

- I can explain each concept in plain language without memorized wording.
- I can draw the lifecycle or data flow from memory.
- I can compare alternatives and state when each is appropriate.
- I can write a small example and discuss complexity and failure cases.
- I can explain what I would measure before optimizing a real system.